STOCK PRICE FORECASTING USING ARIMA AND FOURIER TRANSFORMS

AARON DE LA ROSA

We are going to use of ARIMA and Fourier Transforms as features in a deep learning model for financial forecasting. ARIMA (Autoregressive Integrated Moving Average) is a widely used time-series analysis technique that can help predict future values based on past performance. Fourier Transforms, on the other hand, are a mathematical technique that can be used to analyze time-series data by breaking it down into its component parts, including different frequencies.

With the use of powerful techniques like ARIMA and Fourier Transforms, combined with popular technical indicators, we can make accurate predictions and gain a competitive edge in the dynamic world of finance.

ARIMA

After pre-processing our financial data, we can begin calculating ARIMA (Autoregressive Integrated Moving Average). ARIMA is a powerful time series analysis technique used to forecast future values based on past data. The model works by identifying patterns in the data and using them to make predictions. We will start by using ARIMA to generate predictions for Goldman Sachs stock prices.

A statistical model is autoregressive if it predicts future values based on past values. For example, an ARIMA model might seek to predict a stock's future prices based on its past performance or forecast a company's earnings based on past periods.

To select the optimal ARIMA parameters, we will use the auto_arima function from the pmdarima library. This function automatically selects the best ARIMA model parameters using a stepwise approach. We will set the seasonal parameter to False since we are not working with seasonal data. The trace parameter is set to True so that we can see the output of the function.

We'll see auto_arima results that the best model parameters are (0,1,0). We observe that the ARIMA predictions are very similar to the actual closing prices. This suggests that the model is doing a good job of capturing the underlying patterns in the data and making accurate predictions.

Fourier Transforms

In addition to using ARIMA, we can also incorporate Fourier Transforms into our predictive model to analyze and forecast time series data. Fourier transforms a mathematical technique that decomposes a signal into its underlying frequencies. By analyzing the frequency components of a signal, we can identify patterns and trends that may be difficult to see in the original data.

The Fourier transform is a mathematical function that takes a time-based pattern as input and determines the overall cycle offset, rotation speed and strength for every possible cycle in the given pattern.

To apply Fourier transforms to our financial data, we will first calculate the Fourier transform of the closing prices using the NumPy library. We will then plot the Fourier transforms alongside the actual closing prices to visualize the frequency components of the data.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
[51]  1 import yfinance as yf
      2 import pandas as pd
      3 import numpy as np
      4 import matplotlib.pyplot as plt
      5 plt.style.use("dark_background")
      6 # Download data
      7 gs = yf.download("NVDA", start="2019-12-11", end="2023-12-11")
```

```
[**********100%%**********]  1 of 1 completed
```

```
[52]  1 import pandas as pd
      2
      3 # Preprocess data
      4 dataset_ex_df = gs.copy()
      5 dataset_ex_df = dataset_ex_df.reset_index()
      6 dataset_ex_df['Date'] = pd.to_datetime(dataset_ex_df['Date'])
      7 dataset_ex_df.set_index('Date', inplace=True)
      8 dataset_ex_df = dataset_ex_df['Close'].to_frame()
```

```
[53]  1 from pmdarima.arima import auto_arima
      2
      3 # Auto ARIMA to select optimal ARIMA parameters
      4 model = auto_arima(dataset_ex_df['Close'], seasonal=False, trace=True)
      5 print(model.summary())
```

0 s se ejecutó 7:26 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
      3 # Auto ARIMA to select optimal ARIMA parameters
[53]  4 model = auto_arima(dataset_ex_df['Close'], seasonal=False, trace=True)
      5 print(model.summary())
```

```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(0,0,0)[0] intercept   : AIC=6835.988, Time=1.35 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=6828.329, Time=0.07 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=6830.213, Time=0.12 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=6830.211, Time=0.22 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=6829.706, Time=0.09 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=6832.022, Time=1.75 sec

Best model:  ARIMA(0,1,0)(0,0,0)[0] intercept
Total fit time: 3.619 seconds
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                 1006
Model:               SARIMAX(0, 1, 0)   Log Likelihood               -3412.164
Date:                Wed, 13 Dec 2023   AIC                           6828.329
Time:                        01:18:57   BIC                           6838.154
Sample:                             0   HQIC                          6832.062
                                - 1006
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept      0.4186      0.243      1.725      0.084      -0.057       0.894
sigma2        52.0654      0.917     56.756      0.000      50.267      53.863
```

0 s se ejecutó 7:26 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb ☆

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[53]
```
intercept      0.4186      0.243      1.725      0.084      -0.057       0.894
sigma2        52.0654      0.917     56.756      0.000      50.267      53.863
===================================================================================
Ljung-Box (L1) (Q):                   0.12   Jarque-Bera (JB):              6910.57
Prob(Q):                              0.73   Prob(JB):                         0.00
Heteroskedasticity (H):               7.78   Skew:                             1.32
Prob(H) (two-sided):                  0.00   Kurtosis:                        15.57
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

[54]
```
from statsmodels.tsa.arima.model import ARIMA
import numpy as np

# Define the ARIMA model
def arima_forecast(history):
    # Fit the model
    model = ARIMA(history, order=(0,1,0))
    model_fit = model.fit()

    # Make the prediction
    output = model_fit.forecast()
    yhat = output[0]
    return yhat

```

0 s se ejecutó 7:26 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb ☆

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[54]
```
X = dataset_ex_df.values
size = int(len(X) * 0.8)
train, test = X[0:size], X[size:len(X)]

# Walk-forward validation
history = [x for x in train]
predictions = list()
for t in range(len(test)):
    # Generate a prediction
    yhat = arima_forecast(history)
    predictions.append(yhat)
    # Add the predicted value to the training set
    obs = test[t]
    history.append(obs)
```

[55]
```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5), dpi=100)
plt.plot(dataset_ex_df.iloc[size:,:].index, test, label='Real')
plt.plot(dataset_ex_df.iloc[size:,:].index, predictions, color='red', label='Predicted')
plt.title('ARIMA Predictions vs Actual Values')
plt.xlabel('Date')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```

0 s se ejecutó 7:26 p.m.

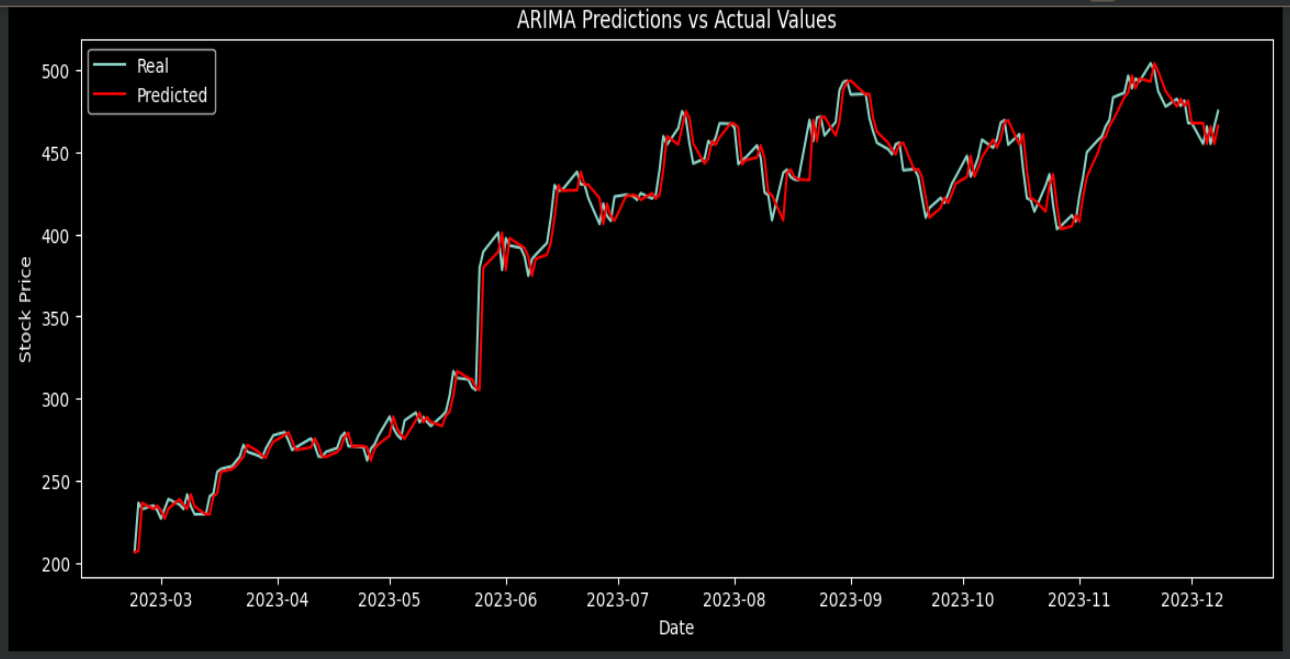
Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI



0 s se ejecutó 7:26 p.m.

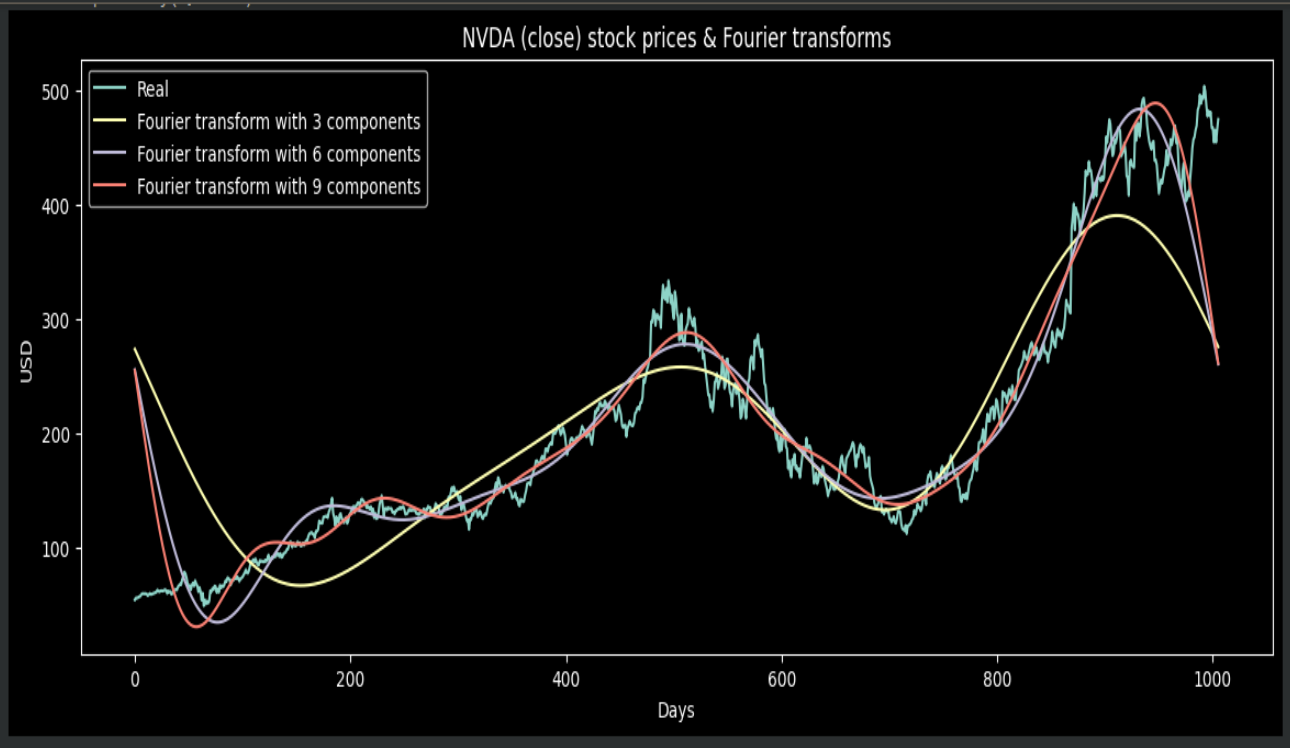
Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI



0 s se ejecutó 7:26 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[58]
```
dataset_ex_df['ema_20'] = ema(dataset_ex_df["Close"], 20)
dataset_ex_df['ema_50'] = ema(dataset_ex_df["Close"], 50)
dataset_ex_df['ema_100'] = ema(dataset_ex_df["Close"], 100)

dataset_ex_df['rsi'] = rsi(dataset_ex_df["Close"])
dataset_ex_df['macd'] = macd(dataset_ex_df["Close"])
dataset_ex_df['obv'] = obv(dataset_ex_df["Close"], dataset_ex_df["Close"])
```

[59]
```
# Create arima DF using predictions
arima_df = pd.DataFrame(history, index=dataset_ex_df.index, columns=['ARIMA'])

# Set Fourier Transforms DF
fft_df.reset_index(inplace=True)
fft_df['index'] = pd.to_datetime(dataset_ex_df.index)
fft_df.set_index('index', inplace=True)
fft_df_real = pd.DataFrame(np.real(fft_df['fft']), index=fft_df.index, columns=['Fourier_real'])
fft_df_imag = pd.DataFrame(np.imag(fft_df['fft']), index=fft_df.index, columns=['Fourier_imag'])

# Technical Indicators DF
technical_indicators_df = dataset_ex_df[['ema_20', 'ema_50', 'ema_100', 'rsi', 'macd', 'obv', 'Close']]

# Merge DF
merged_df = pd.concat([arima_df, fft_df_real, fft_df_imag, technical_indicators_df], axis=1)
merged_df = merged_df.dropna()
merged_df
```

0 s se ejecutó 7:26 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[59]

	ARIMA	Fourier_real	Fourier_imag	ema_20	ema_50	ema_100	rsi	macd	obv	Close
2020-01-02	59.977501	624.663742	3567.781025	57.652661	56.126117	55.334238	79.720841	1.300668	231.407497	59.977501
2020-01-03	59.017502	-34.803948	3590.896291	57.782646	56.239505	55.407174	67.103769	1.246756	172.389996	59.017502
2020-01-06	59.264999	2067.323193	5632.206954	57.923822	56.358152	55.483566	68.157823	1.210053	231.654995	59.264999
2020-01-07	59.982498	-851.209889	4934.346350	58.119887	56.500283	55.572654	69.602130	1.224743	291.637493	59.982498
2020-01-08	60.095001	1253.786272	4548.710149	58.307993	56.641252	55.662205	67.266515	1.231270	351.732494	60.095001
...	...	...	...	...	...	...	...	...	...	...
2023-12-04	455.100006	-4099.545411	-13196.434462	471.697612	459.592106	438.916011	34.691854	5.334960	15882.339844	455.100006
2023-12-05	465.660004	237.250260	-20679.397013	471.122602	459.830063	439.445595	34.820194	4.263875	16347.999847	465.660004
2023-12-06	455.029999	-10558.988250	-35469.511692	469.589973	459.641825	439.754197	33.839386	2.528138	15892.969849	455.029999
2023-12-07	465.959991	30704.466021	-40876.216379	469.244260	459.889597	440.273124	36.859847	2.011327	16358.929840	465.959991
2023-12-08	475.059998	9226.077977	-32189.239489	469.798140	460.484514	440.961973	42.343183	2.309424	16833.989838	475.059998

992 rows × 10 columns

[60]
```
merged_df.plot()
```

<Axes: >

0 s se ejecutó 7:26 p.m.

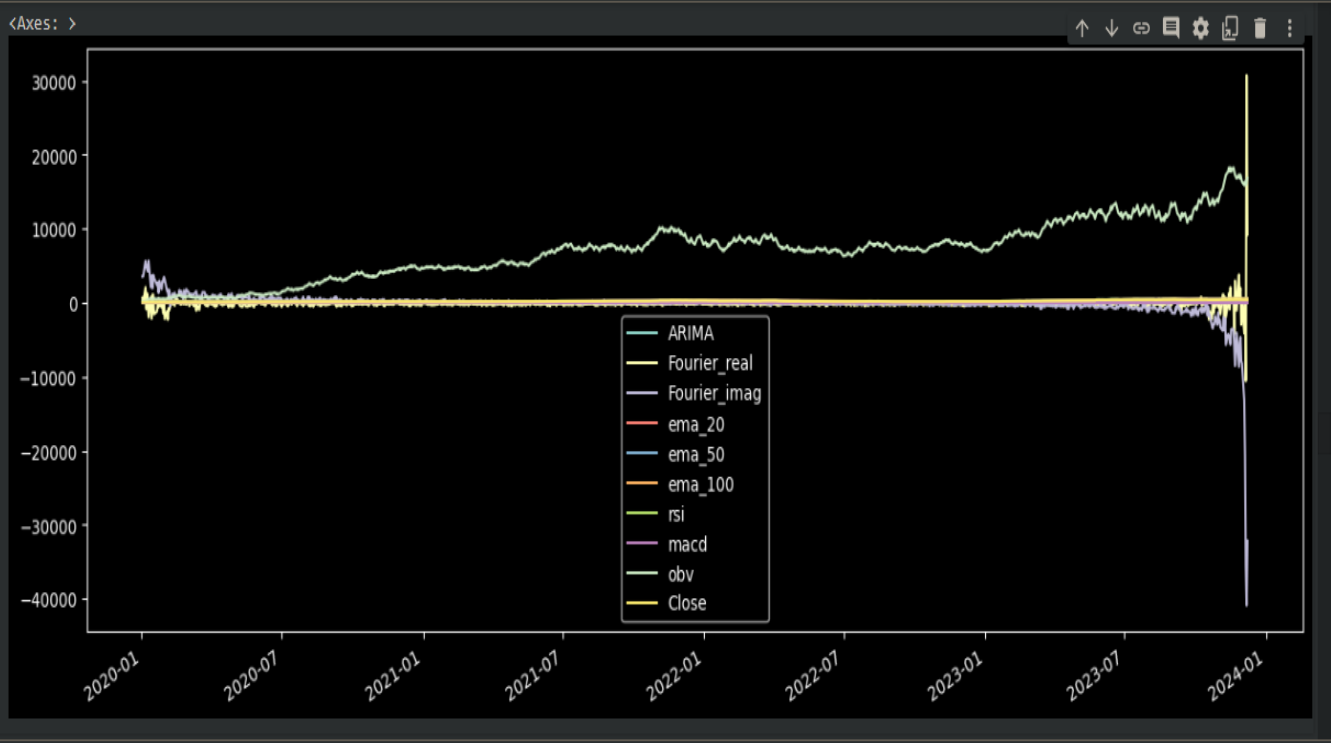
Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI



✓ 0 s se ejecutó 7:38 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
[66]  1 # Import keras modules
      2 from keras.models import Sequential
      3 from keras.layers import Dense
      4 from keras.callbacks import EarlyStopping
      5
      6 # Define model
      7 model = Sequential()
      8 model.add(Dense(32, activation='relu', input_dim=X_train.shape[1]))
      9 model.add(Dense(16, activation='relu'))
     10 model.add(Dense(1, activation='linear'))
     11
     12 # Compile the model
     13 model.compile(optimizer='adam', loss='mse')
     14
     15
     16 # Define the early stopping callback
     17 early_stop = EarlyStopping(monitor='val_loss', patience=20, verbose=1, mode='min')
     18
     19 # Train the model with early stopping callback
     20 history = model.fit(X_train, y_train, epochs=800, batch_size=32, verbose=1, validation_data=(X_test, y_test), callbacks=[early_stop], shuffle=False)
```

```
25/25 [==============================] - 0s 6ms/step - loss: 7.0282 - val_loss: 64.2503
Epoch 720/800
25/25 [==============================] - 0s 6ms/step - loss: 7.0041 - val_loss: 64.0716
Epoch 721/800
25/25 [==============================] - 0s 5ms/step - loss: 6.9806 - val_loss: 63.8972
```

✓ 0 s se ejecutó 7:38 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
[66] 25/25 [==============================] - 0s 5ms/step - loss: 6.7363 - val_loss: 63.1607
     Epoch 743/800
     25/25 [==============================] - 0s 5ms/step - loss: 6.7343 - val_loss: 63.4522
     Epoch 744/800
     25/25 [==============================] - 0s 6ms/step - loss: 6.7307 - val_loss: 63.7970
     Epoch 745/800
     25/25 [==============================] - 0s 6ms/step - loss: 6.7246 - val_loss: 64.1967
     Epoch 746/800
     25/25 [==============================] - 0s 8ms/step - loss: 6.7153 - val_loss: 64.6520
     Epoch 747/800
     25/25 [==============================] - 0s 6ms/step - loss: 6.7017 - val_loss: 65.1603
     Epoch 748/800
     25/25 [==============================] - 0s 8ms/step - loss: 6.6830 - val_loss: 65.7178
     Epoch 749/800
     25/25 [==============================] - 0s 8ms/step - loss: 6.6587 - val_loss: 66.3167
     Epoch 750/800
     25/25 [==============================] - 0s 6ms/step - loss: 6.6283 - val_loss: 66.9478
     Epoch 751/800
     25/25 [==============================] - 0s 5ms/step - loss: 6.5918 - val_loss: 67.5989
     Epoch 752/800
     25/25 [==============================] - 0s 6ms/step - loss: 6.5495 - val_loss: 68.2547
     Epoch 753/800
     25/25 [==============================] - 0s 5ms/step - loss: 6.5022 - val_loss: 68.8986
     Epoch 754/800
     25/25 [==============================] - 0s 3ms/step - loss: 6.4510 - val_loss: 69.5127
     Epoch 755/800
     25/25 [==============================] - 0s 5ms/step - loss: 6.3976 - val_loss: 70.0787
     Epoch 755: early stopping
```

✓ 0 s se ejecutó 7:38 p.m.

Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
[67] 16
     17 print(f"Mean Squared Error (MSE): {mse}")
     18 print(f"Mean Absolute Error (MAE): {mae}")
     19 print(f"R2 Score: {r2}")
     20 print(f"Explained Variance Score: {evs}")
     21 print(f"Mean Absolute Percentage Error (MAPE): {mape}")
     22 print(f"Mean Percentage Error (MPE): {mpe}")

     7/7 [==============================] - 0s 4ms/step
     Mean Squared Error (MSE): 70.07877104583748
     Mean Absolute Error (MAE): 5.0100443471017195
     R2 Score: 0.9903339432273264
     Explained Variance Score: 0.9919672903434962
     Mean Absolute Percentage Error (MAPE): 26.606164372287104
     Mean Percentage Error (MPE): -5.0983582979338715
```

```
[68] 1 # Plot final Predictions
     2 plt.figure(figsize=(14, 5), dpi=100)
     3 plt.plot(test_scaled_df.index, y_test, label='Real')
     4 plt.plot(test_scaled_df.index, y_pred, color='red', label='Predicted')
     5 plt.title('Model Predictions vs Real Prices')
     6 plt.xlabel('Date')
     7 plt.ylabel('Stock Price')
     8 plt.legend()
     9 plt.show()
```

✓ 0 s se ejecutó 7:38 p.m.

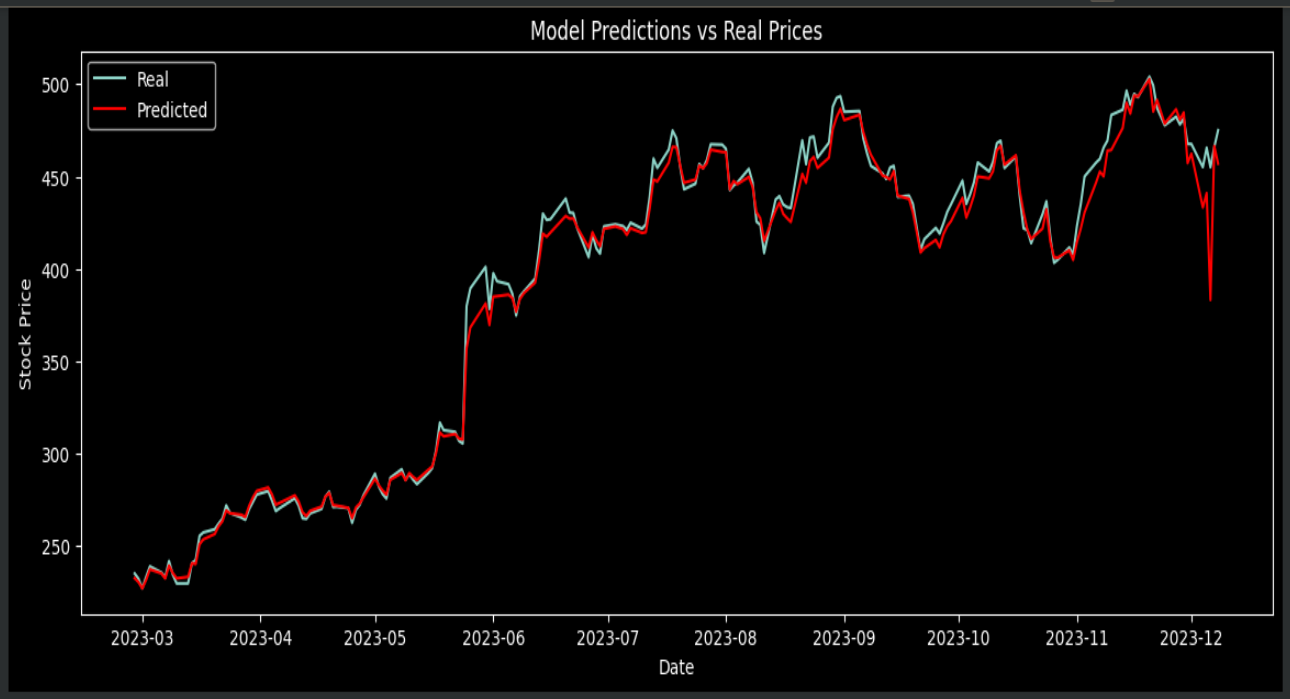
Predicting Stock Prices using ARIMA, Fourier Transforms.ipynb ☆

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI



✓ 0 s se ejecutó 7:38 p.m.